Flutter Provider Tutorial: Complete Guide

Table of Contents

1. Introduction
2. What is Provider?
3. Project Structure
4. Understanding the Code
5. How Provider Works
6. Advanced Patterns
7. Best Practices
8. Common Errors

Introduction

This tutorial demonstrates state management in Flutter using the **Provider** package. We'll build a simple app that manages user data across multiple screens, showing how Provider simplifies state sharing without passing data through constructors.

What We're Building

- A home page displaying the current username
- A settings page to change the username
- Shared state across both pages using Provider

What is Provider?

Provider is Flutter's recommended state management solution. It's built on top of `InheritedWidget` but much easier to use.

Why Provider?

Problem	Solution with Provider
Passing data through multiple widgets	Access data anywhere in the widget tree
Rebuilding entire widget tree	Only rebuild widgets that need updates
Complex state management	Simple, intuitive API
Tight coupling between widgets	Separation of UI and business logic

Key Concepts

1. **ChangeNotifier**: A class that can notify listeners when data changes
2. **ChangeNotifierProvider**: Makes a ChangeNotifier available to the widget tree
3. **MultiProvider**: Provides multiple providers at once

4. **Consumer/context.watch**: Rebuilds when data changes
5. **context.read**: Access data without listening to changes

---

Project Structure

```
lib/
├── main.dart                    # App entry point
├── providers/
│   └── user_provider.dart       # State management logic
└── pages/
    ├── homepage.dart            # Displays username
    └── settings.dart            # Updates username
```

Understanding the Code

1. Setup (pubspec.yaml)

First, add the Provider dependency:

```yaml
dependencies:
  flutter:
    sdk: flutter
  provider: ^6.1.5+1
```

Run `flutter pub get` to install.

---

2. Creating the Provider (user_provider.dart)

```dart
import 'package:flutter/material.dart';

class UserProvider extends ChangeNotifier {

  // State variable
  String username;

  // Constructor with default value
  UserProvider({
    this.username = "Guest",
  });

  // Method to update state
  void changeUsername({required String newUsername}) async {
    username = newUsername;
    notifyListeners();  // ⭐ Key: Notify all listeners of change
```

```
  }
}
```

Breaking It Down:

`extends ChangeNotifier`

- Gives the class the ability to notify widgets when data changes
- Like a radio station that can broadcast updates

Named Parameters with Curly Braces `{}`

```
// Named parameter - must specify parameter name when calling
void changeUsername({required String newUsername})

// Usage:
changeUsername(newUsername: "John")  // ✓ Correct
changeUsername("John")               // ✗ Wrong
```

`notifyListeners()`

- The magic method that tells all listening widgets to rebuild
- Without this, UI won't update even though data changed
- Think of it as "announcing" the change

---

3. Providing the State (main.dart)

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import 'package:provider_flutter/pages/homepage.dart';
import 'package:provider_flutter/providers/user_provider.dart';

void main() {
  runApp(const MainApp());
}

class MainApp extends StatelessWidget {
  const MainApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MultiProvider(
      providers: [
        ChangeNotifierProvider(
          create: (context) => UserProvider(),
        ),
      ],
      child: MaterialApp(
```

```
        debugShowCheckedModeBanner: false,
        home: HomePage(),
      ),
    );
  }
}
```

Key Points:

`MultiProvider`

- Wraps the entire app to make providers available everywhere
- Can hold multiple providers (for larger apps)
- Place it high in the widget tree (usually wrapping MaterialApp)

`ChangeNotifierProvider`

- Creates and provides a ChangeNotifier
- `create` callback: Creates the provider instance once
- Instance is shared across all widgets below

Provider Scope

- Any widget inside the `child` can access UserProvider
- Widgets outside cannot access it

---

4. Consuming State - Reading (homepage.dart)

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import 'package:provider_flutter/pages/settings.dart';
import 'package:provider_flutter/providers/user_provider.dart';

class HomePage extends StatefulWidget {
  const HomePage({super.key});

  @override
  State<HomePage> createState() => _HomePageState();
}

class _HomePageState extends State<HomePage> {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text("Provider Tutorial"),
      ),
      body: Center(
        // ⭐ context.watch() - Rebuilds when username changes
        child: Text(context.watch<UserProvider>().username),
```

```
      ),
      bottomNavigationBar: BottomNavigationBar(
        items: [
          BottomNavigationBarItem(icon: Icon(Icons.home), label: "Home"),
          BottomNavigationBarItem(icon: Icon(Icons.settings), label: "Settings"),
        ],
        onTap: (index) {
          if (index == 1) {
            Navigator.push(
              context,
              MaterialPageRoute(builder: (context) => Settings()),
            );
          }
        }
      ),
    );
  }
}
```

`context.watch<UserProvider>()`

- **Listens** to changes in UserProvider
- **Rebuilds** the widget when `notifyListeners()` is called
- Use when you need the UI to update automatically

```
Text(context.watch<UserProvider>().username)
```

This Text widget will automatically show the new username when it changes.

---

5. Consuming State - Writing (settings.dart)

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import 'package:provider_flutter/providers/user_provider.dart';

class Settings extends StatelessWidget {
  final TextEditingController textcontroller = TextEditingController();

  Settings({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: Center(
        child: TextFormField(
          decoration: InputDecoration(
            labelText: "Enter new username",
          ),
```

```
            controller: textcontroller,
          ),
        ),
        floatingActionButton: FloatingActionButton(
          onPressed: () {
            // ⭐ context.read() - Access provider without listening
            context.read<UserProvider>().changeUsername(
              newUsername: textcontroller.text
            );
          },
          child: Icon(Icons.save),
        ),
      );
    }
  }
```

`context.read<UserProvider>()`

- **Accesses** the provider without listening
- **Doesn't rebuild** when data changes
- Use when you only need to call methods (like button clicks)

```
context.read<UserProvider>().changeUsername(newUsername: "John")
```

How Provider Works

The Complete Flow

```
1. User enters new username in Settings page
   ↓
2. Clicks save button
   ↓
3. context.read<UserProvider>() accesses the provider
   ↓
4. changeUsername() method is called
   ↓
5. username is updated
   ↓
6. notifyListeners() broadcasts change
   ↓
7. HomePage's context.watch() receives notification
   ↓
8. HomePage Text widget rebuilds with new username
   ↓
9. User sees updated username
```

Visual Diagram

```
┌──────────────────────────────────────────┐
│         MultiProvider (Root)             │
│  ┌──────────────────────────────────┐    │
│  │      UserProvider Instance       │    │
│  │  username: "Guest" → "John"      │    │
│  │  notifyListeners() ──────────┐   │    │
│  └──────────────────────────────────┘    │
└──────────────────────────────────────────┘
              ↓                   ↓
     ┌────────────────┐   ┌────────────────┐
     │   HomePage     │   │   Settings     │
     │                │   │                │
     │ context.watch  │   │ context.read   │
     │ → Listens ✓    │   │ → No listen ✗  │
     │ → Rebuilds ✓   │   │ → Just calls   │
     └────────────────┘   └────────────────┘
```

Advanced Patterns

1. Multiple State Variables

```
class UserProvider extends ChangeNotifier {
  String username;
  String email;
  int age;

  UserProvider({
    this.username = "Guest",
    this.email = "guest@example.com",
    this.age = 0,
  });

  void updateProfile({
    String? newUsername,
    String? newEmail,
    int? newAge,
  }) {
    if (newUsername != null) username = newUsername;
    if (newEmail != null) email = newEmail;
    if (newAge != null) age = newAge;
    notifyListeners();
  }
}
```

2. Using Consumer Widget

Alternative to `context.watch()` for better performance:

```
// Only rebuilds the Consumer, not the entire widget
Scaffold(
  body: Center(
    child: Consumer<UserProvider>(
      builder: (context, userProvider, child) {
        return Text(userProvider.username);
      },
    ),
  ),
)
```

3. Selector for Precise Rebuilds

Only rebuild when specific property changes:

```
// Only rebuilds when username changes, not other properties
Selector<UserProvider, String>(
  selector: (context, provider) => provider.username,
  builder: (context, username, child) {
    return Text(username);
  },
)
```

4. Multiple Providers

```
return MultiProvider(
  providers: [
    ChangeNotifierProvider(create: (_) => UserProvider()),
    ChangeNotifierProvider(create: (_) => ThemeProvider()),
    ChangeNotifierProvider(create: (_) => CartProvider()),
  ],
  child: MaterialApp(
    home: HomePage(),
  ),
);
```

5. Async Operations

```
class UserProvider extends ChangeNotifier {
  String username = "Guest";
  bool isLoading = false;

  Future<void> fetchUserFromApi() async {
```

```
    isLoading = true;
    notifyListeners(); // Update UI to show loading

    try {
      // Simulate API call
      await Future.delayed(Duration(seconds: 2));
      username = "John Doe";
    } catch (e) {
      username = "Error";
    } finally {
      isLoading = false;
      notifyListeners(); // Update UI with result
    }
  }
}
```

6. Disposing Resources

```
class UserProvider extends ChangeNotifier {
  late StreamSubscription _subscription;

  UserProvider() {
    _subscription = someStream.listen((data) {
      // Handle data
      notifyListeners();
    });
  }

  @override
  void dispose() {
    _subscription.cancel(); // Clean up
    super.dispose();
  }
}
```

Best Practices

☑ DO

1. **Keep providers focused**

```
// Good: Single responsibility
class UserProvider extends ChangeNotifier { }
class CartProvider extends ChangeNotifier { }

// Bad: Too many responsibilities
class AppProvider extends ChangeNotifier {
  String username;
```

```
  List<Product> cart;
  ThemeData theme;
  // ... too much!
}
```

2. **Use context.read() for actions**

```
// Good: No unnecessary rebuilds
onPressed: () => context.read<UserProvider>().save()

// Bad: Rebuilds button on every change
onPressed: () => context.watch<UserProvider>().save()
```

3. **Use context.watch() for display**

```
// Good: Updates when data changes
Text(context.watch<UserProvider>().username)
```

4. **Use named parameters**

```
// Good: Clear and readable
changeUsername(newUsername: "John")

// Bad: What does "John" represent?
changeUsername("John")
```

5. **Call notifyListeners() after state changes**

```
void updateData() {
  data = newData;
  notifyListeners(); // Always notify!
}
```

❌ DON'T

1. **Don't use context.watch() in callbacks**

```
// Bad: Creates unnecessary rebuilds
onPressed: () {
  final provider = context.watch<UserProvider>();
  provider.save();
}
```

---

2. **Don't forget the generic type**

```
// Bad: Results in error
context.read()

// Good: Specify the type
context.read<UserProvider>()
```

3. **Don't put providers too low in the tree**

```
// Bad: Can't access in other parts of app
class HomePage extends StatelessWidget {
  Widget build(context) {
    return ChangeNotifierProvider( // Too low!
      create: (_) => UserProvider(),
      child: ...,
    );
  }
}
```

4. **Don't call notifyListeners() in constructor**

```
// Bad: Can cause issues
UserProvider() {
  username = "Guest";
  notifyListeners(); // Don't do this!
}
```

Common Errors

1. "Tried to call Provider.of"

Error:

```
Failed assertion: 'T != dynamic': Tried to call Provider.of<dynamic>
```

Cause: Missing type parameter

```
context.read() // ✗ Wrong
```

Fix:

```
context.read<UserProvider>() // ✓ Correct
```

---

2. "Could not find the correct Provider"

Error:

```
Could not find the correct Provider<UserProvider> above this Widget
```

Cause: Trying to access provider outside its scope

Fix: Ensure ChangeNotifierProvider is above the widget in the tree:

```
// Wrong order
MaterialApp(
  home: HomePage(), // Can't access provider here
)
Provider(...) // Too low!

// Correct order
Provider(
  child: MaterialApp(
    home: HomePage(), // Can access provider here
  ),
)
```

---

3. "UI Not Updating"

Cause: Forgot to call `notifyListeners()`

```
// Wrong
void changeUsername({required String newUsername}) {
  username = newUsername; // UI won't update!
}

// Correct
void changeUsername({required String newUsername}) {
  username = newUsername;
  notifyListeners(); // Now UI updates!
}
```

---

4. "The method 'MultiProvider' isn't defined"

Cause: Missing import

Fix:

```
import 'package:provider/provider.dart';
```

Testing Your Provider

```
import 'package:flutter_test/flutter_test.dart';
import 'package:provider_flutter/providers/user_provider.dart';

void main() {
  test('Username changes correctly', () {
    // Arrange
    final provider = UserProvider();
    expect(provider.username, 'Guest');

    // Act
    provider.changeUsername(newUsername: 'John');

    // Assert
    expect(provider.username, 'John');
  });

  test('notifyListeners is called', () {
    // Arrange
    final provider = UserProvider();
    var notified = false;
    provider.addListener(() => notified = true);

    // Act
    provider.changeUsername(newUsername: 'Jane');

    // Assert
    expect(notified, true);
  });
}
```

Summary

Key Takeaways

1. **Provider = Shared State**: Access data anywhere without passing it through constructors
2. **ChangeNotifier = State Container**: Holds data and notifies listeners of changes
3. **context.watch() = Listen & Rebuild**: Use in UI to automatically update
4. **context.read() = Access Only**: Use in callbacks/actions without listening

5. **notifyListeners() = Trigger Update**: Must call after state changes
6. **Named Parameters** `{}`: Make code readable and maintainable

The Three Steps to Provider

1. **Create** a ChangeNotifier class
2. **Provide** it using ChangeNotifierProvider
3. **Consume** it with context.watch() or context.read()

When to Use Provider

- ✓ Sharing state across multiple widgets
- ✓ User preferences (theme, language)
- ✓ Authentication state
- ✓ Shopping cart
- ✓ Form data across screens
- ✓ App-wide settings

When NOT to Use Provider

- ✗ Local widget state (use StatefulWidget instead)
- ✗ Single-use data (just pass it as parameter)
- ✗ Extremely complex state (consider Riverpod or Bloc)

Next Steps

1. **Add more features**: Try adding email, profile picture, etc.
2. **Persist data**: Save username using SharedPreferences
3. **Add validation**: Check username length, special characters
4. **Multiple providers**: Create ThemeProvider for dark/light mode
5. **Error handling**: Show snackbar when save fails
6. **Learn Riverpod**: The next evolution of Provider

Useful Resources

- Provider Package
- Flutter State Management
- ChangeNotifier API

Happy Coding! 🚀